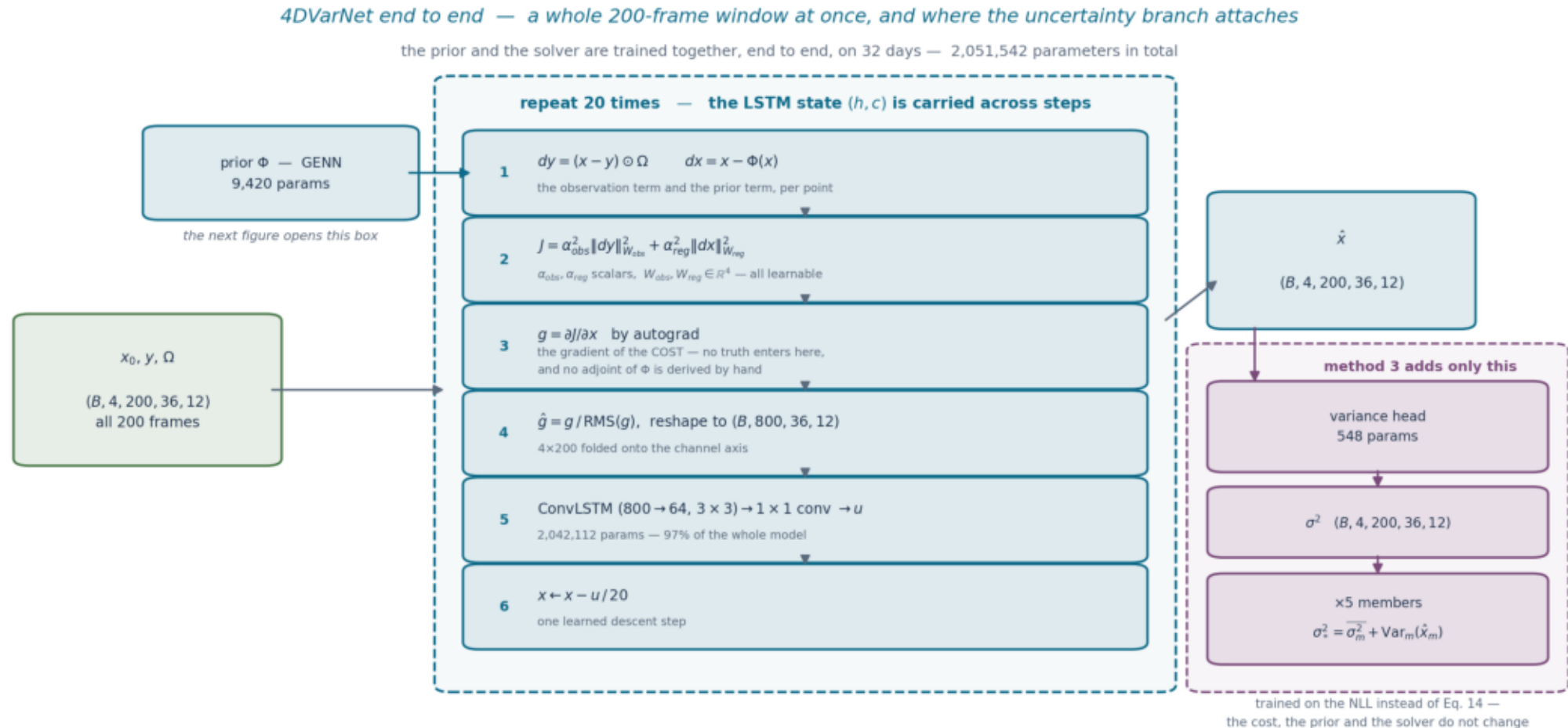


Three methods on the same observations

EnKF · 4DVarNet · 4DVarNet with predictive uncertainty

Xinle Zhang

The pipeline, end to end

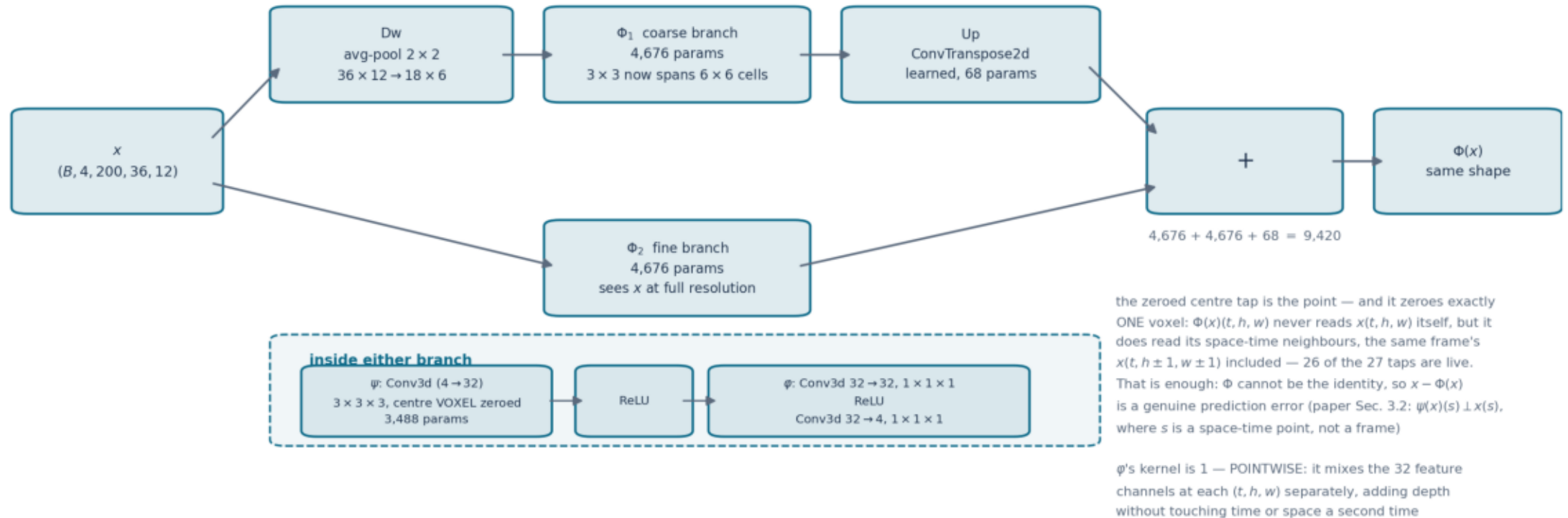


- Method 2 is everything up to $\hat{x}$: minimise a cost with two terms — fit the observations, and stay consistent with a learnt dynamical prior — over a whole 200-frame window at once, in 20 learned descent steps.
- Method 3 is the branch on the right: a 548-parameter head after the solve, five members, and a different training loss. Nothing above it changes.

Inside method 2 — the prior is where the dynamics live

Inside the prior Φ — 9,420 parameters, 0.5% of the model, and the only part that encodes the dynamics

$$\Phi(x) = \text{Up}(\Phi_1(\text{Dw}(x))) + \Phi_2(x) \quad (\text{paper Eq. 10})$$

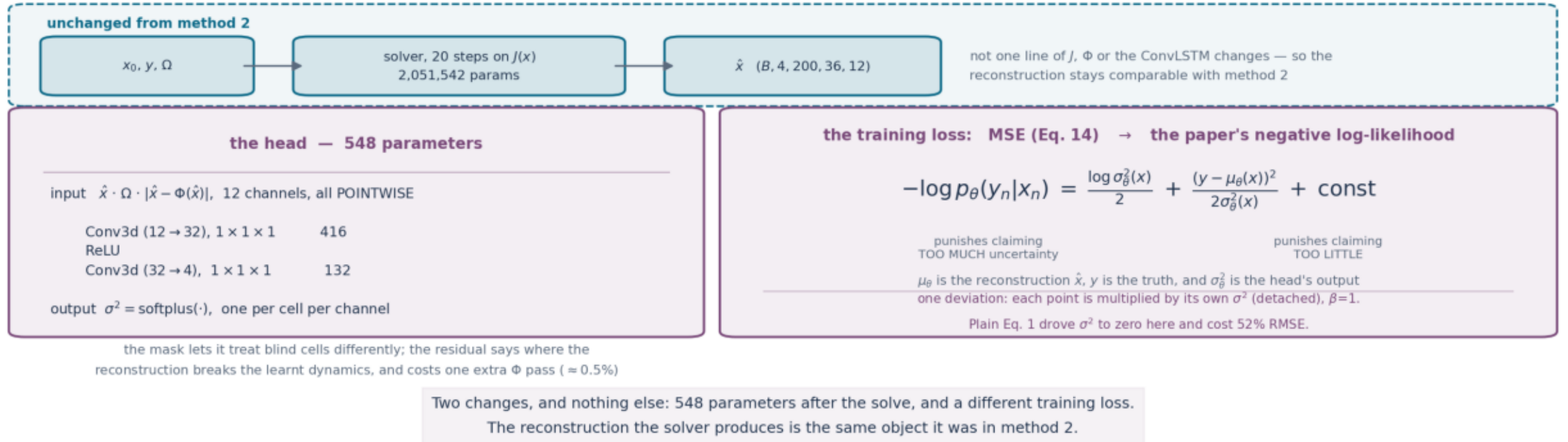


- Two branches summed (paper Eq. 10): a coarse one on the half-resolution grid, where a 3×3 kernel reaches 6×6 original cells, and a fine one on the grid itself. 9,420 parameters in total.
- The kernel's centre tap is held at zero, so a cell is never used to predict itself — 26 of the 27 taps stay live, including the same frame's spatial neighbours. That is already enough to stop Φ collapsing to the identity.

Method 3 — the same solver, asked for its uncertainty

Method 3, training time — the cost J , the prior and the solver are untouched; the head's 548 parameters and the loss are the whole change

σ^2 stays OUTSIDE $J(x)$ on purpose: it is not part of the physical state being assimilated, so the gradient handed to the solver is still the gradient of the cost (4DVarNet Sec. 3.4)

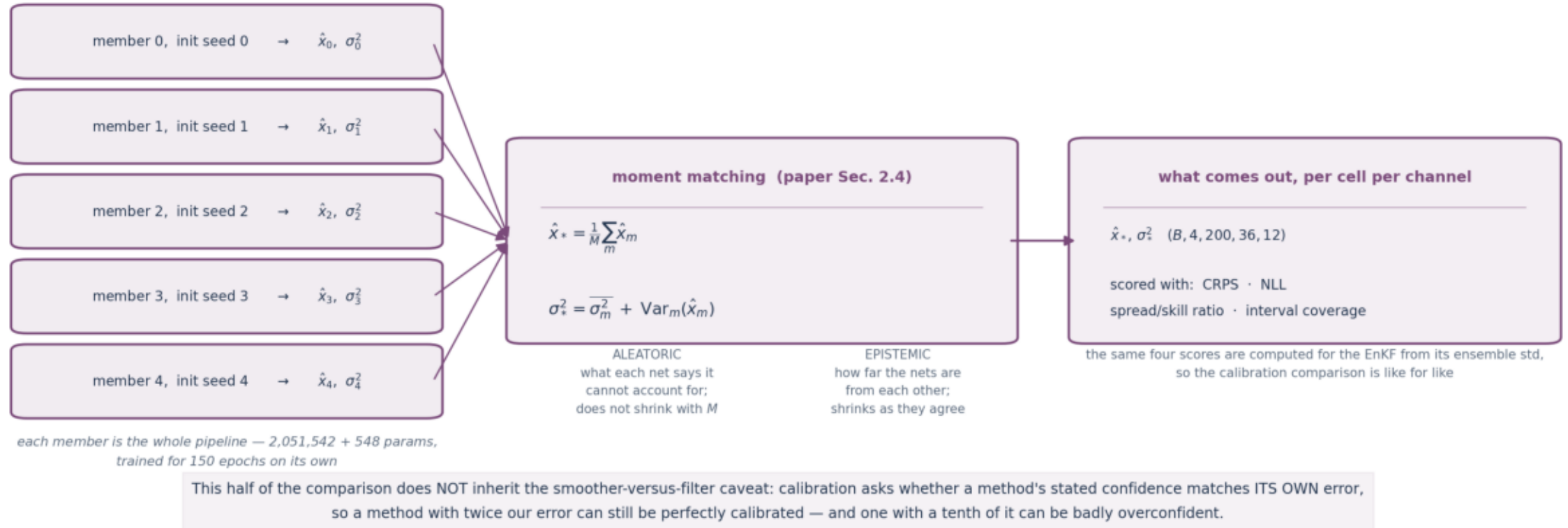


- Two changes: a 548-parameter head that reads the finished reconstruction and outputs a per-cell variance, and the Gaussian NLL in place of squared error. The cost, the prior and the solver are untouched, so the reconstruction stays comparable.

Method 3 at inference — five members, two variance terms

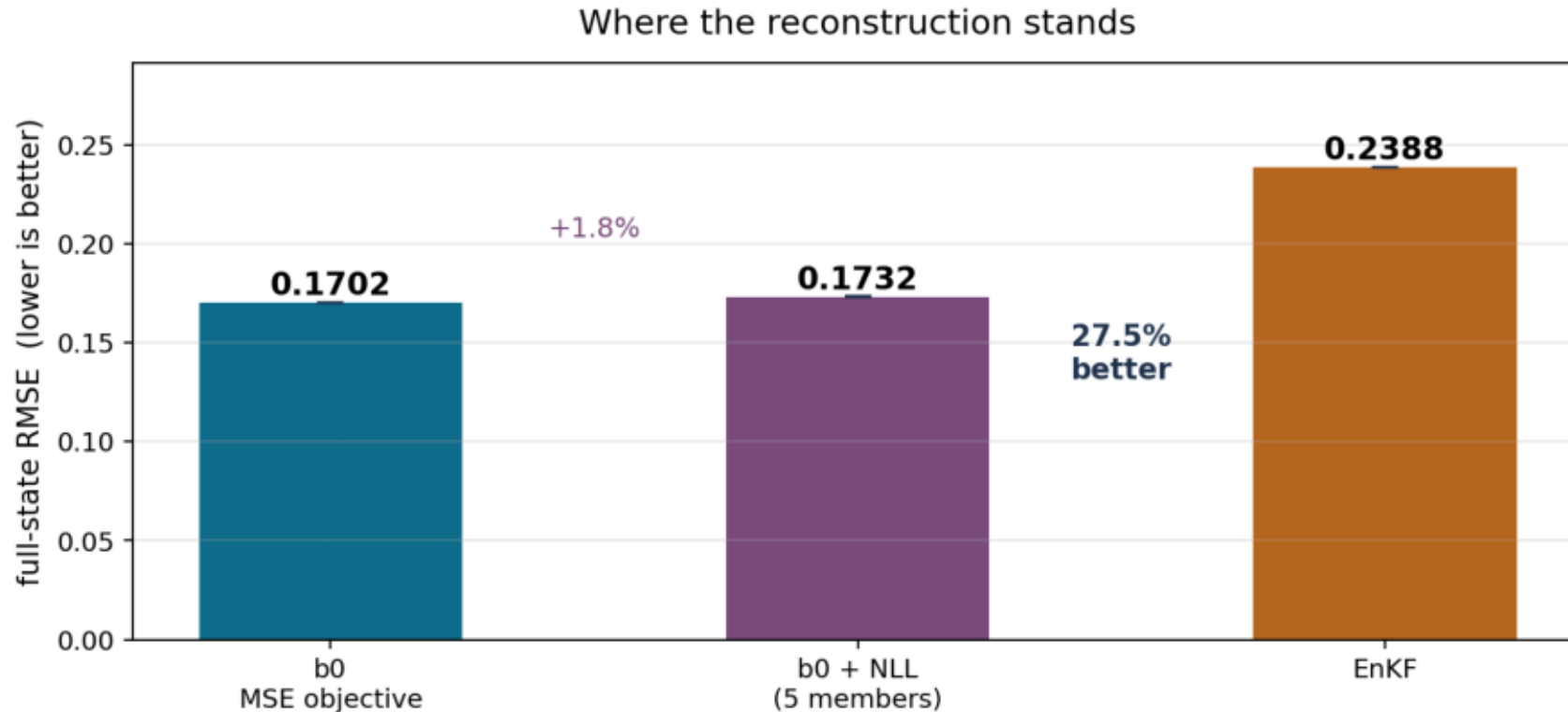
Method 3, inference time — five members, and where each half of σ_^2 comes from*

the five differ in ONE thing: the weight initialisation. Same 32 days, same robot routes, same noise realisation, same window order (paper Sec. 2.4 — bagging is reported to hurt)



- The five members differ in exactly one thing — the weight initialisation. Same days, same robot routes, same noise realisation, same window order, which is what the paper's Sec. 2.4 prescribes.
- Their means agree to 0.3% on RMSE, so the epistemic term is small in aggregate — but it is not optional: the learnt sigma alone scores 0.65 spread/skill against 0.87 for the five together.

Result — reconstruction accuracy



Same 7 test days and the same convention for all three. The error bar on the middle bar is the spread across the 5 independently initialised members (std 0.0005, i.e. 0.3% — they converge to almost the same solution). The EnKF's configuration is the original project's and was not retuned.

- Full-state RMSE on the held-out days: 0.1702 for 4DVarNet, 0.1732 ± 0.0005 with the uncertainty head, 0.2388 for the filter — 27.5% better.
- Two things are NOT matched and both favour us: our solver reconstructs frame t from the whole 200-frame window including frames after t , while the filter only sees up to t ; and our prior and solver are trained on 32 days of this data while the filter's surrogate was given to us pre-trained.

Appendix — the variational cost, written out

The variational cost, with the channel sum written out

$J(x) =$

$\alpha_{obs}^2 \cdot \frac{1}{N} (w_{obs,0}^2 \Sigma(dy_0)^2 + w_{obs,1}^2 \Sigma(dy_1)^2 + w_{obs,2}^2 \Sigma(dy_2)^2 + w_{obs,3}^2 \Sigma(dy_3)^2)$

$+ \alpha_{reg}^2 \cdot \frac{1}{N} (w_{reg,0}^2 \Sigma(dx_0)^2 + w_{reg,1}^2 \Sigma(dx_1)^2 + w_{reg,2}^2 \Sigma(dx_2)^2 + w_{reg,3}^2 \Sigma(dx_3)^2)$

fit the observations

obey the learnt dynamics

the two residuals	the sums, and N
$dy_c = (x_c - y_c) \odot \Omega_c$ observation misfit zero wherever a cell was not observed, so a blind cell reaches J only through dx $dx_c = x_c - \Phi(x)_c$ prior misfit how far the state is from what the learnt dynamics say it should be	Σ runs over every point of the window: 200 frames $\times$ 36 $\times$ 12 cells = 86,400 per channel the 200 frames are simply ADDED IN — the time axis is treated exactly like the two space axes $N = T \cdot H \cdot W = 86,400$ (the same number) so each Σ/N is a mean square, and J does not grow with the window length

$c = 0 \dots 3$ are the 4 physical channels. α_{obs} and α_{reg} are scalars, w_{obs} and w_{reg} are one weight per channel — all four are LEARNED, not set by hand.

c	channel	$w_{obs,c}$	$w_{reg,c}$
0	density	1.248	0.956
1	vx	1.063	1.099
2	vy	1.075	0.986
3	var	1.930	0.790
$\alpha_{obs} = 1.090$ $\alpha_{reg} = 0.911$			

The weights exist because the 4 channels are on different scales — velocity swings by a couple of m/s where density is a small count. A single unweighted sum of squares would be a statement about vx alone.

Appendix — how the prior's convolution computes one value

How ψ computes ONE value — the sum, then the same sum on real numbers

$$\psi_o(t, h, w) = \sum_{c=1}^4 \sum_{\tau=-1}^1 \sum_{i=-1}^1 \sum_{j=-1}^1 M[\tau, i, j] W[o, c, \tau, i, j] x[c, t+\tau, h+i, w+j] + b_o$$

M is the mask: 1 everywhere except $M[0, 0, 0] = 0$. Below: feature 3 at voxel (41, 10, 6) of atc-20130811, checked against conv3d in checks/diag_channel_budget.py.

the density channel at $\tau = 0$ (the same frame), written out

input x				weight W				product			
0.211	1.030	0.127		+0.0036	-0.1150	+0.0550		+0.0008	-0.1185	+0.0070	
1.327	—	0.995	×	+0.2156	—	+0.2116	=	+0.2861	—	+0.2105	sum
0.823	1.356	1.087		+0.0433	+0.2410	+0.0979		+0.0356	+0.3267	+0.1064	= +0.8546

the centre is blanked by the mask. The value there is 2.2973, and the unmasked weight would have contributed +0.2210 — one term out of 104.

the same thing for every channel and every frame offset — 4×3 partial sums

	$\tau = -1$	$\tau = 0$	$\tau = +1$	per channel
density	+0.6149	+0.8546	-0.0506	+1.4189
vx	+0.2750	+0.1856	+0.1481	+0.6087
vy	+0.0279	+0.0061	+0.0302	+0.0642
var	+0.0039	+0.0718	+0.0306	+0.1062

+ bias -0.2673

$\psi_3 = +1.9307$

ReLU → +1.9307 (a negative ψ would be zeroed)

what one output value costs

reads $4 \times 26 = 104$ input values

weights $32 \times 4 \times 27 = 3,456$

bias 32

of those, 128 are held at zero

⇒ 3,328 live weights

padding (1, 1, 1): at $t=1$, $t=200$

and on the grid border the missing neighbours are read as ZERO

then ϕ — kernel 1

at each (t, h, w) separately:

$32 \rightarrow 32$, ReLU, $32 \rightarrow 4$

a per-voxel MLP: it adds depth but cannot reach across time or space, so no amount of it can put back a dependence on the predicted cell

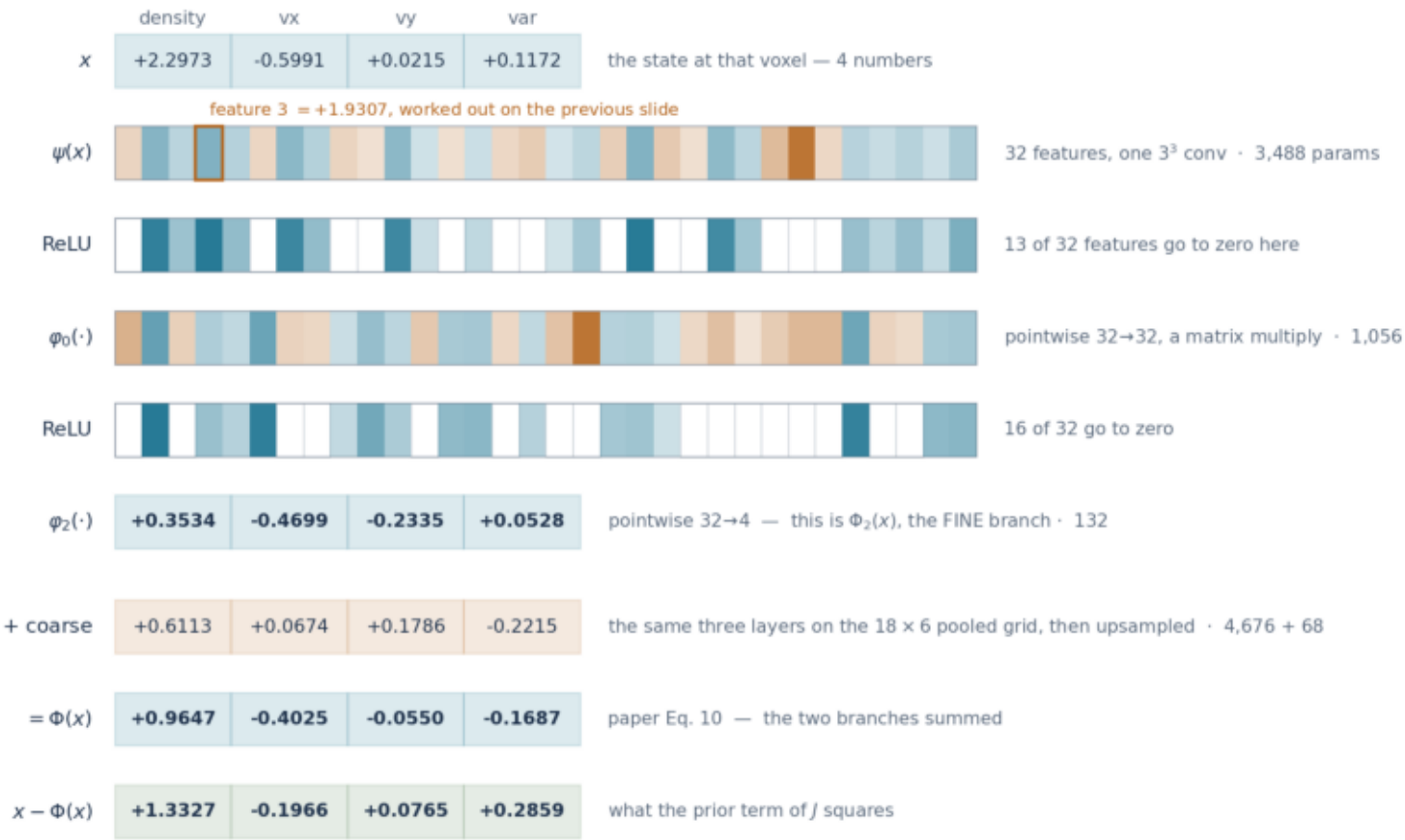
the next slide follows all three layers at this same voxel

So Φ cannot be the identity — that is all the mask has to buy. It is NOT the stronger claim that Φ ignores frame t : $x(t, h \pm 1, w \pm 1)$ is read freely, and 26/27 of the stencil is live. The paper's Sec. 3.2 wording is $\psi(x)(s) \perp x(s)$, with s a space-time POINT.

Appendix — the whole prior, at one voxel

The whole prior, at one voxel — $(t, h, w) = (41, 10, 6)$ of atc-20130811, following x all the way to $\Phi(x)$

a branch is THREE weight layers, not one: ψ then two pointwise convs, with a ReLU after each of the first two. Φ is two such branches plus a learned upsampling.



the three layers, per branch

ψ	Conv3d (4→32), 3^3	3,488
φ_0	Conv3d (32→32), 1^3	1,056
φ_2	Conv3d (32→4), 1^3	132
		4,676

×2 branches 9,352
+ learned upsampling 68
= Φ 9,420

ψ is the ONLY layer that reaches across time and space. Both φ layers have kernel 1, so at a fixed voxel they are just 32×32 and 4×32 matrix multiplies — no amount of depth after ψ can reintroduce a dependence on the cell being predicted.

Both the fine branch and Φ itself are asserted against the module's own forward in checks/diag_channel_budget.py, so these rows cannot drift from what the model computes.

Appendix — from the gates to the update

From the gates to the update — what each of the four decides, then the cell, then the read-out



the gate convolution's 256 outputs are four groups of 64, and each group has a different job

	gate	squash	decides
<i>i</i>	input	σ	how much of what this step just computed is written into memory
<i>f</i>	forget	σ	how much of the PREVIOUS step's memory survives
$\tilde{g}$	candidate	$\tanh$	the content this step computed, before any gating
<i>o</i>	output	σ	how much of the memory is read out as the update

each is (B, 64, 36, 12) — one value per hidden channel per cell — and each is its own 7,776-term convolution over the same 864 input channels

the cell, at hidden channel 49 of voxel (41, 10, 6)

$c_{prev} = +0.945930$ inherited from step 9

$f = 0.999541$
 $\tilde{g} = -0.307852$

$i = 0.970749$
 $o = 1.000000$

$c \leftarrow f c_{prev} + i \tilde{g} = 0.9995 \times +0.9459 + 0.9707 \times -0.3079$
 $\quad = +0.646650$

$h \leftarrow o \tanh(c) = 1.0000 \times \tanh(+0.6466) = +0.569410$

the read-out, and what it moves

$u = \sum_{k=1}^{64} W_{out}[r, k] h[k]$ over all 64 hidden channels, no bias
 $r = c \cdot T + t = 41$ (density, $t = 41$)

this channel's share: $W_{out}[r, 49] = -0.1196$, $\times h = -0.068091$
all 64 terms: $u = -0.018383$

$x \leftarrow x - u/20$: $+0.006827 - -0.000919 = +0.007746$

one step moves this cell by 13.5% of its value

h and *c* are (64, 36, 12) and are carried across the 20 DESCENT STEPS, not across physical time. The 800 axis is *c*·*T* + *t* — the 4 channels and 200 frames flattened together, which is why *dT* is fixed by the weight shape. 2,042,112 parameters in all: 1,990,912 in the gate convolution, 51,200 in the read-out.

Appendix — the variance head, step by step

The variance head, step by step — both convolutions have kernel 1, so at one voxel each is a matrix-vector product

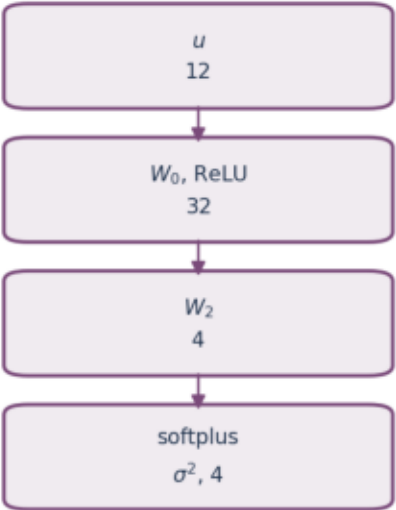
$3 \times 4 = 12 \rightarrow 32 \rightarrow 4$, then $\sigma^2 = \text{obs_var} + \text{softplus}(\cdot)$. 548 parameters. Voxel (41, 10, 6), member 0.

	density	vx	vy	var	density	vx	vy	var	density	vx	vy	var
$U =$	+2.2580	-0.4236	-0.1350	+0.1257	+1.0000	+1.0000	+1.0000	+1.0000	+3.1982	+0.0505	+0.2078	+0.2465
	$\hat{x}$				Ω				$ \hat{x} - \Phi(\hat{x}) $			

the 12 inputs: the reconstruction, the mask, and the prior residual

layer 1, hidden unit 10 — all 12 terms				
input	channel	u	W_0	product
$\hat{x}$	density	+2.2580	+1.0857	+2.4516
	vx	-0.4236	+0.0004	-0.0002
	vy	-0.1350	-0.0750	+0.0101
	var	+0.1257	+0.4280	+0.0538
Ω	density	+1.0000	-0.2910	-0.2910
	vx	+1.0000	-0.1247	-0.1247
	vy	+1.0000	-0.1210	-0.1210
	var	+1.0000	-0.1798	-0.1798
$ \hat{x} - \Phi(\hat{x}) $	density	+3.1982	+1.1204	+3.5834
	vx	+0.0505	+0.1663	+0.0084
	vy	+0.2078	+0.2387	+0.0496
	var	+0.2465	-0.5642	-0.1391
sum				+5.3011
+ b_0				+0.1307
z, then ReLU				+5.4318
all four Ω products are negative — being observed pushes this unit DOWN				

layer 2 → density — 32 terms
16 of the 32 hidden units are 0 at this voxel, so 16 terms are live
$\Sigma W_2[0, h] a[h] = -6.8714$ $+ b_2 = -2.8839$
$v = -9.7552$
then softplus
$\sigma^2 = \text{softplus}(v)$
$\text{softplus}(-9.7552) = 0.000058$ $\sigma = 0.00762$
nothing floors this — a very negative v means the head is claiming confidence and has to be right about it



416 + 132 = 548 params
in the whole head